Project Report

Smart Kitchen Assistant Model Development

A B.Tech Project

Project Guide:	Prof. Ganesh Bhimanna Bagler
Student Name:	Satwik Garg
Roll Number:	2022461
Project Domain:	Conversational AI for Smart Kitchens
Type:	Voice-based Intelligent Assistant

Submitted in partial fulfillment of academic requirements

for the Bachelor of Technology degree

Department of Engineering

2025–2026

Declaration

I hereby declare that this report titled *Smart Kitchen Assistant Model Development* is an original documentation of the project work carried out under the guidance of **Prof. Ganesh Bhimanna Bagler**. The work presented here is based on implementation, experimentation, and system analysis conducted for this B.Tech project.

To the best of my knowledge, this report does not contain any material previously submitted for the award of any degree or diploma, except where due acknowledgment has been made through references.

Student Name: Satwik Garg
Roll Number: 2022461

Abstract

This report presents the design and implementation of a **Smart Kitchen Assistant** that enables hands-free recipe interaction using voice. The system combines wake-word detection, voice activity detection, automatic speech recognition, semantic recipe retrieval, intent classification, conversational response generation, and text-to-speech playback into one end-to-end loop tailored for kitchen environments.

The assistant is built as a modular Python pipeline. An offline data pipeline prepares recipe chunks, searchable text, ingredient dictionaries, and vector embeddings. At runtime, the assistant listens continuously for the wake phrase "Hey Cook", captures user commands, classifies intents, retrieves recipe context, and generates spoken responses. The architecture supports navigation commands such as next, previous, go-to-step, repeat ingredients, and question answering with contextual grounding.

The implemented prototype integrates multiple fallback strategies, including local ASR and in-memory session storage, to improve robustness under API or connectivity failures. Experimental observations from the implemented system indicate practical response times and stable behavior for a 100-recipe pilot dataset, establishing a strong baseline for future expansion toward larger datasets, multilingual support, and richer kitchen task automation.

Methodologically, the project emphasizes explicit module boundaries and data contracts across all stages, enabling independent debugging and iterative improvement of subsystems without breaking end-to-end behavior. This includes structured session state representation, deterministic navigation transitions, and chunk-wise speech generation for interruption-aware playback.

The resulting system demonstrates that a student-scale implementation can still achieve production-inspired properties such as graceful degradation, modular extensibility, and practical response latency for procedural voice guidance. The report concludes with outcomes, limitations, and a roadmap toward larger-scale deployment and empirical user studies.

Acknowledgement and Scope of Work

Acknowledgement. I express sincere gratitude to **Prof. Ganesh Bhimanna Bagler** for valuable guidance, continuous feedback, and encouragement throughout the project. The mentorship provided direction in both technical execution and scientific framing of this work. I also acknowledge all open-source communities and API providers whose tools enabled rapid prototyping and evaluation of the assistant.

Scope of Work. The completed project scope includes:

- Design and implementation of an end-to-end voice assistant loop for cooking use-cases.
- Construction of an offline data pipeline for recipe ingestion, chunking, normalization, dictionary creation, and embedding generation.
- Development of intent-driven runtime orchestration for recipe search, step navigation, and question answering.
- Integration of session-aware conversational flow with resumable audio playback and interruption handling.
- Documentation of architecture, schemas, algorithms, design decisions, and evaluation observations.

Contents

Declaration **i**

Abstract **ii**

Acknowledgement and Scope of Work **iii**

Table of Contents **vi**

1 Introduction **1**

1.1 Background and Motivation 1

1.2 Problem Statement 1

1.3 Research Questions 2

1.4 Objectives 2

1.5 Representative User Scenario 3

1.6 Key Contributions 3

1.7 System Requirements 3

1.8 Project Scope 4

1.9 Significance of the Work 4

1.10 Report Organization 5

2 Literature Review **6**

2.1 Review Scope 6

2.2 Voice Interaction Systems 6

2.3 Speech Recognition and Robustness 7

2.4 Semantic Retrieval in Recipe Search 7

2.5 LLMs for Conversational Cooking Assistance 7

2.6 Session Persistence and Dialogue Continuity 8

2.7 Audio Output and Human Factors . . . 8
2.8 Comparative Design Criteria . . . 8
2.9 Technology Comparison . . . 9
2.10 Research Gap Addressed . . . 10
2.11 Summary . . . 10

3 Methodology **11**
3.1 System Overview . . . 11
3.2 Offline Data Pipeline . . . 12
3.2.1 Offline Flow Logic . . . 12
3.2.2 Chunk Construction Strategy . . . 13
3.3 Runtime Interaction Pipeline . . . 13
3.3.1 Interaction Cycle . . . 13
3.4 Core Module Design . . . 14
3.4.1 Wake Word and Voice Activity Detection . . . 14
3.4.2 ASR Layer . . . 14
3.4.3 Intent Classification Layer . . . 14
3.5 Intent-Driven Dispatch Strategy . . . 14
3.5.1 Navigation State Model . . . 15
3.6 Retrieval and Ranking . . . 15
3.6.1 Response Generation and Audio Streaming . . . 16
3.7 Session and State Management . . . 16
3.8 Error Handling and Fallbacks . . . 16
3.9 Implementation Notes . . . 17

4 Results and Discussion **18**
4.1 Implementation Status . . . 18
4.2 Module Completion Snapshot . . . 19
4.3 Observed Pipeline Performance . . . 19
4.4 Interaction Behavior Analysis . . . 20
4.5 Example End-to-End Runtime Trace . . . 20
4.6 Functional Outcome . . . 21
4.7 Failure-Mode and Recovery Summary . . . 21
4.8 Scalability Considerations . . . 21

4.9 Discussion 22
4.10 Validity Considerations 22
4.11 Conclusion 23

Chapter 1

Introduction

1.1 Background and Motivation

Voice interfaces are increasingly useful in environments where hands and visual attention are occupied. Cooking is one such setting, where users often need quick guidance without touching devices. A smart kitchen assistant must therefore be responsive, interruption-tolerant, and context-aware.

This project, **Smart Kitchen Assistant Model Development**, builds a practical voice-first assistant that can retrieve recipes, narrate ingredients and steps, answer cooking questions, and navigate instructions through natural commands.

The motivation of this work comes from the mismatch between current recipe-consumption interfaces and real cooking workflows. In a practical kitchen setting, users frequently switch between ingredient preparation, heat control, and time-bound execution of steps. This creates two usability requirements:

- interaction should be **low-friction** (minimal manual device usage),
- guidance should be **state-aware** (assistant remembers recipe context).

This project addresses both through a conversational architecture with persistent session state and interruptible spoken output.

1.2 Problem Statement

Conventional recipe applications are primarily screen-driven and require frequent manual interaction. During cooking, this causes friction and interruptions. The core problem addressed in this work is:

How to design and implement a robust, real-time, conversational recipe assistant that supports hands-free interaction in noisy kitchen environments.

From an engineering viewpoint, the problem decomposes into four tightly coupled sub-problems:

1. detecting user intent quickly from noisy and short commands,
2. retrieving correct recipe content even when the query is paraphrased,
3. managing dialogue and step position across interruptions,
4. generating natural responses fast enough to preserve conversation flow.

1.3 Research Questions

The implementation and analysis in this report are guided by the following research questions:

- **RQ1:** Can a modular voice pipeline provide reliable step-wise cooking guidance with minimal latency?
- **RQ2:** Does semantic retrieval improve recipe search quality for natural spoken queries compared to strict keyword matching?
- **RQ3:** How effectively can fallback mechanisms preserve usability under API or service degradation?
- **RQ4:** Can session persistence support practical, interruption-tolerant cooking conversations?

1.4 Objectives

The project objectives are summarized in table 1.1.

Table 1.1: Project objectives and implementation focus

ID	Objective	Implementation focus
O1	Hands-free activation and command capture	Wake-word detection + VAD-based recording
O2	Reliable speech understanding	Hybrid ASR with API-first and local fallback
O3	Accurate recipe discovery	Semantic retrieval using embeddings and vector search
O4	Guided recipe execution	Intent-based navigation and session tracking
O5	Natural spoken responses	LLM response generation with chunked TTS playback

1.5 Representative User Scenario

figure 1.1 illustrates a typical interaction sequence.

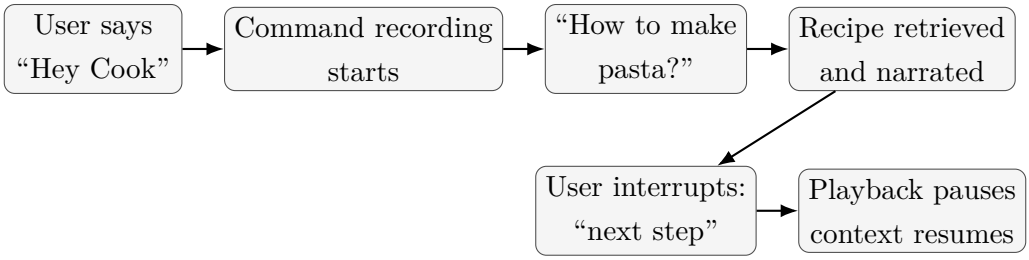

Figure 1.1: Typical user interaction trajectory in the Smart Kitchen Assistant.

1.6 Key Contributions

The implemented system contributes:

- A modular architecture separating wake word, ASR, intent classification, retrieval, navigation, LLM reasoning, session state, and TTS.
- A complete offline-to-online pipeline, from recipe preprocessing to runtime conversational interaction.
- Robustness through layered fallbacks (ASR, intent, storage) and interruption-aware audio handling.
- Structured recipe response handling suitable for step tracking and future multi-turn personalization.

1.7 System Requirements

The requirements were derived from the target environment (kitchen), expected command style (short spoken phrases), and safety of interaction continuity.

Table 1.2: Functional and non-functional requirements

ID	Requirement	Type	Design response
R1	Wake-word activation without button press	Functional	Always-on Porcupine listener thread
R2	Short-command capture with auto-stop	Functional	VAD recording using RMS silence detection
R3	Accurate recipe retrieval from natural queries	Functional	Embedding-based semantic search (Milvus)
R4	Step navigation commands	Functional	Intent dispatcher + navigator state machine
R5	Resume after interruption	Functional	Session manager + pausable chunked audio player
R6	End-to-end conversational responsiveness	Non-functional	Rule-first intent path + streaming TTS
R7	Service-level fault tolerance	Non-functional	Cascading ASR/session/response fallbacks
R8	Maintainable research prototype architecture	Non-functional	Modular Python components and explicit schemas

1.8 Project Scope

The current prototype targets English voice interactions with a pilot dataset of 100 recipes. It supports recipe search, start, next/previous step, jump-to-step, repeat, repeat ingredients, pause/resume, and recipe-question answering.

The scope intentionally prioritizes system reliability and integration over broad feature breadth. Therefore:

- multimodal sensing (vision, IoT devices) is not included,
- nutrition recommendation optimization is not modeled,
- full production hardening (authentication, distributed tracing, autoscaling) is outside current scope.

1.9 Significance of the Work

This project is significant in two dimensions. First, it demonstrates a complete voice-AI pipeline for a concrete domain with operational constraints. Second, it provides reusable engineering artifacts

(schemas, component interfaces, data pipeline, and runtime orchestration) that can support future academic and product-oriented work in conversational assistants for task-centric environments.

1.10 Report Organization

This report is organized as follows:

- **Chapter 2** reviews relevant literature and technology choices.
- **Chapter 3** presents system methodology, architecture, and implementation.
- **Chapter 4** summarizes outcomes, observations, and limitations.

Chapter 2

Literature Review

2.1 Review Scope

The literature and system references considered in this chapter are selected around six core axes of a kitchen-domain voice assistant:

1. wake-word and low-latency activation,
2. speech recognition in noisy environments,
3. semantic retrieval for recipe matching,
4. intent understanding and conversational orchestration,
5. speech synthesis for long-form guidance,
6. session continuity under interruptions.

2.2 Voice Interaction Systems

Modern voice assistants combine multiple subsystems: keyword spotting, speech recognition, intent understanding, response planning, and speech synthesis. For robust interaction, each stage should tolerate upstream uncertainty and variable audio quality.

In this project, Porcupine is used for wake-word detection due to low-latency local inference [1]. The architecture follows a practical "always listening + event-driven dispatch" pattern widely used in speech applications.

The major design trade-off in wake-word selection is between customizability and deterministic behavior. Fully trainable KWS models can optimize custom phrases but increase model-management

complexity. In contrast, prebuilt wake-word engines offer faster integration and stable inference-time behavior, which is suitable for student-scale implementation and iterative system-level experimentation.

2.3 Speech Recognition and Robustness

Automatic speech recognition (ASR) quality strongly affects downstream intent and retrieval performance. Whisper demonstrates strong robustness to diverse speech conditions [2]. Cloud ASR services often provide lower latency in production settings, while local ASR gives offline fallback.

Accordingly, this assistant adopts a hybrid strategy: Deepgram ASR as primary and local Whisper as fallback [3, 2].

This hybrid approach is aligned with practical reliability engineering principles: primary services maximize quality and speed, while secondary paths preserve continuity during outages or quota failures. For kitchen use-cases, continuity is critical because interactions are procedural and interruptions often occur mid-step.

2.4 Semantic Retrieval in Recipe Search

Keyword-only search can fail when users use paraphrases or synonyms. Sentence-level embeddings improve semantic matching [4]. Vector databases operationalize this approach for fast nearest-neighbor retrieval at runtime.

This project uses `all-MiniLM-L6-v2` embeddings with Milvus vector search for recipe retrieval, followed by recipe detail enrichment via backend APIs [4, 5].

Embedding retrieval is particularly useful for cooking queries because user phrasing is heterogeneous:

- "how to make aloo curry",
- "show me potato curry recipe",
- "what can I cook with potatoes".

These may not share exact keywords but remain semantically proximate in embedding space.

2.5 LLMs for Conversational Cooking Assistance

Large language models are useful for generating conversational responses, clarifications, and structured step-wise narration. In production-style pipelines, LLM outputs are safer when constrained to

schema-driven formats. The implemented system leverages Gemini-based generation and fallback classification in low-confidence intent cases [6].

Another key observation from recent conversational system design is that LLMs are strongest when used with explicit role constraints:

- as a fallback classifier for ambiguity resolution,
- as a bounded natural-language formatter over retrieved factual context,
- as a short-turn conversational layer over deterministic navigation state.

This reduces hallucination risk compared to unconstrained end-to-end generation.

2.6 Session Persistence and Dialogue Continuity

Cooking interactions are naturally multi-turn. Session storage must track progress state, spoken sections, and dialogue history. Redis-style key-value persistence is commonly used for low-latency session handling and TTL-based expiration [7]. This work uses Upstash Redis with in-memory fallback for resilience.

Persisted session state also enables context restoration after temporary interruptions. In voice-first systems, these interruptions are common and include user pauses, repeated wake-word invocations, or command repairs (for example, "repeat that" immediately after a step was spoken).

2.7 Audio Output and Human Factors

Speech synthesis for task guidance differs from short-answer TTS use-cases. Recipe narration requires:

- clear sentence boundaries,
- controllable pacing,
- resumability and chunk-level playback control.

Deepgram TTS is used for naturalness and fast API turnaround [3]. The project addresses long-response usability by chunking output into sentence-level units.

2.8 Comparative Design Criteria

table 2.1 maps major design criteria to architecture-level decisions.

Table 2.1: Design criteria and corresponding architectural choices

Criterion	System decision	Justification
Low command latency	Rule-first intent classification	Avoids unnecessary LLM calls for common navigation commands
Interrupt tolerance	Persistent wake thread + pausable playback	Supports "always-interruptible" cooking dialogue
Semantic understanding	Embedding retrieval via Milvus	Handles paraphrased recipe search better than strict keywords
Operational resilience	Multi-layer fallbacks	Maintains usability during external service failures
Maintainability	Modular components and explicit schemas	Easier debugging, upgrades, and academic reproducibility

2.9 Technology Comparison

table 2.2 summarizes the major stack decisions.

Table 2.2: Technology choices and rationale

Layer	Chosen	Alternatives	Reason for selection
Wake word	Porcupine	PocketSphinx, custom KWS	Low-latency local trigger and reliable fixed wake phrase
ASR	Deepgram + Whisper fallback	Whisper-only, Google STT	Fast cloud path with offline resilience
Retrieval	SentenceTransformer + Milvus	BM25/keyword search	Better semantic matching for paraphrased recipe queries
LLM	Gemini Flash family	GPT/Llama variants	Good latency-cost trade-off and JSON-friendly generation
Session state	Upstash Redis (+ memory fallback)	SQLite only, in-memory only	Multi-turn persistence with graceful degradation
TTS	Deepgram Aura voice	Local TTS engines	More natural speech for step-by-step cooking guidance

2.10 Research Gap Addressed

Most single-component demonstrations (ASR-only, TTS-only, or chatbot-only) do not solve full kitchen interaction constraints: wake-word interruption, step navigation, persistence, and semantic retrieval under real-time expectations. This project addresses that integration gap by implementing the full pipeline with explicit fallbacks and state tracking.

2.11 Summary

The literature indicates that no single model or API solves kitchen-assistant requirements in isolation. Practical systems require careful coupling of deterministic components (wake detection, navigation state, session persistence) with probabilistic components (ASR, LLM generation, semantic retrieval). This motivates the hybrid architecture implemented in this project and detailed in the next chapter.

Chapter 3

Methodology

3.1 System Overview

The system is implemented as two connected pipelines:

- **Offline data pipeline**: recipe fetch, chunking, normalization, dictionary creation, embedding generation.
- **Runtime voice pipeline**: wake-word → recording → ASR → intent dispatch → retrieval/navigation/LLM → TTS playback.

The methodology follows an integration-first approach:

1. define stable data contracts between modules,
2. implement each component with explicit fallback behavior,
3. validate complete end-to-end command cycles,
4. refine timing, chunking, and state transitions for conversational flow.

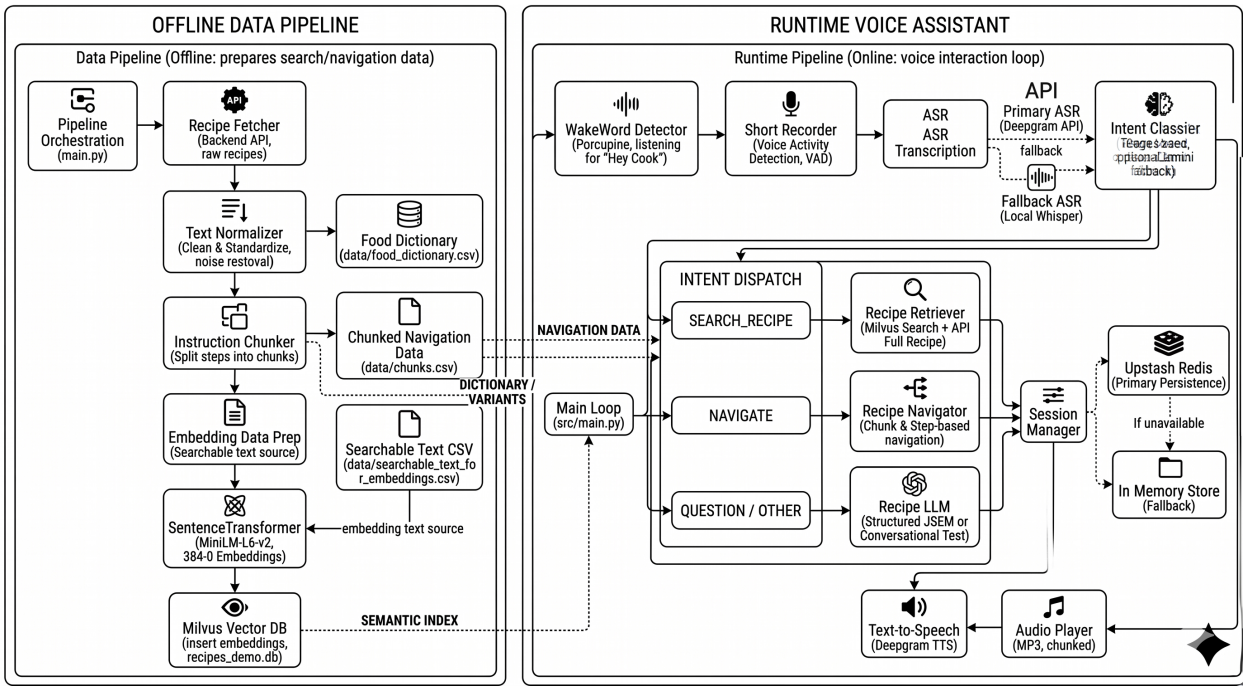

Figure 3.1: End-to-end architecture of the Smart Kitchen Assistant (offline data preparation + runtime interaction loop).

3.2 Offline Data Pipeline

The offline workflow is orchestrated by `data_pipeline/main.py`. For the current pilot configuration, recipes are fetched page-wise and transformed into searchable assets.

Table 3.1: Offline pipeline stages

Step	Module	Output
1	`api_call.py` + `chunker.py`	`data/recipes.csv`, `data/chunks.csv`
2	`normalizer.py`	`data/searchable_text_for_embeddings.csv`
3	`food_dictionary.py`	`data/food_dictionary.csv`
4	`embeddings.py`	Milvus vectors in `recipes_demo.db`

3.2.1 Offline Flow Logic

The pipeline can be interpreted as a deterministic transformation chain:

raw API recipes → chunked instructions → normalized searchable text → embeddings + vector index.

To preserve recoverability, recipe processing is performed incrementally in batches and written to

CSV outputs after each batch cycle. This minimizes loss during partial failures.

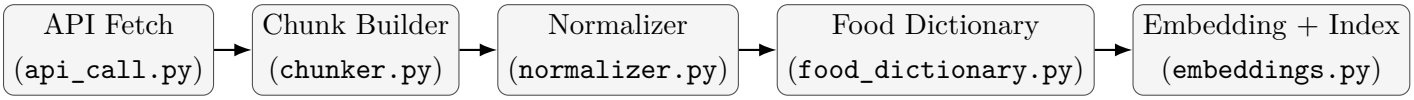

Figure 3.2: Offline data preparation sequence before runtime deployment.

3.2.2 Chunk Construction Strategy

Recipe chunking is used to improve retrieval granularity and navigation control. Let an instruction sequence be $\{s_1, s_2, \ldots, s_n\}$. Chunks are created with bounded word windows $[W_{\min}, W_{\max}]$ and overlap o, producing chunk set $\{c_1, \ldots, c_m\}$ where each chunk preserves a contiguous step range.

3.3 Runtime Interaction Pipeline

At runtime, the assistant follows an event-driven loop implemented in `src/main.py`. Wake-word detection runs continuously on a daemon thread; once detected, the foreground pipeline executes command capture and response generation.

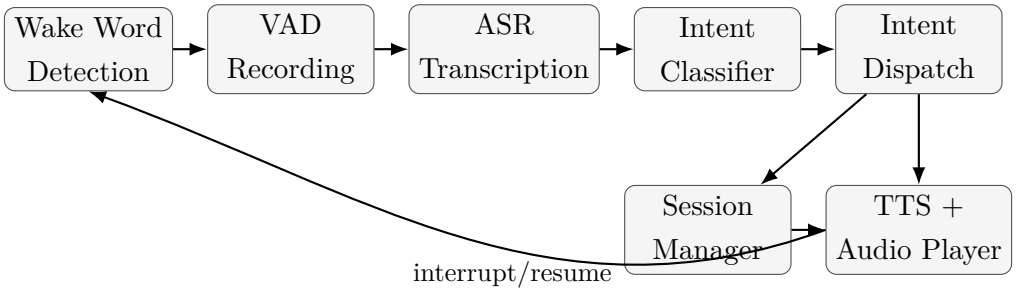

Figure 3.3: Runtime control flow with interruption-aware playback and session updates.

3.3.1 Interaction Cycle

Each interaction cycle follows:

1. Wake word event is triggered.
2. Active playback is paused immediately if running.
3. Voice command is recorded with VAD termination.
4. ASR produces transcription text.
5. Intent classifier predicts intent and entities.

6. Dispatcher routes to retrieval, navigation, QA, or playback control.
7. Response text is converted to chunked audio and streamed.
8. Session state is updated and persisted.

3.4 Core Module Design

3.4.1 Wake Word and Voice Activity Detection

Wake-word detection is handled by a dedicated daemon thread, while recording and silence endpoint detection are executed on the main interaction thread. For audio frame $x = \{x_1, \ldots, x_N\}$, RMS energy is:

$$\mathrm{RMS}(x) = \sqrt{\frac{1}{N}\sum_{i=1}^{N} x_i^2}. \tag{3.1}$$

Recording stops when RMS remains below a configured threshold for a configured duration window.

3.4.2 ASR Layer

The ASR layer uses Deepgram as primary service and Whisper as fallback. After transcription, optional phonetic/fuzzy correction is applied using domain vocabulary (recipe titles and ingredient terms) to improve downstream intent and retrieval performance.

3.4.3 Intent Classification Layer

Intent classification combines:

- rule-based regex patterns with calibrated confidence scores,
- LLM fallback when confidence $< \tau$ (threshold).

This preserves low-latency handling of common commands while retaining flexibility for ambiguous queries.

3.5 Intent-Driven Dispatch Strategy

The intent classifier combines regex-based rules with optional LLM fallback for low-confidence inputs. Supported intents include recipe search, step navigation, repeat, pause/resume, help, and question answering.

Table 3.2: Primary intent groups and handlers

Intent group	Representative intents	Handler path
Recipe discovery	`search_recipe`, `start_recipe`	Retriever + Navigator session initialization
Navigation	`nav_next`, `nav_prev`, `nav_go_to`, `nav_repeat`	Navigator + SessionManager updates
Conversational QA	`question`, `small_talk`, `clarify`	LLM generation with context/history
Playback control	`stop_pause`, `resume`, `confirm`, `cancel`	Audio player state transitions

3.5.1 Navigation State Model

The navigator tracks two linked positions:

- global `step_index`,
- chunk-level `chunk_index`.

State updates are deterministic and mirrored to session storage to allow pause/resume and interruption-aware continuation.

Table 3.3: Representative session fields used during runtime

Field	Type	Purpose
`recipe_id`, `recipe_title`	string	Identifies active recipe context
`step_index`, `chunk_index`	integer	Tracks current navigation position
`current_section`	string	Distinguishes ingredients/steps/done sections
`conversation_history`	list	Stores recent user-assistant turns
`steps_spoken`, `ingredients_spoken`	list	Supports repeat and continuation control
`paused`, `last_intent`	boolean/string	Runtime playback and dialogue state

3.6 Retrieval and Ranking

Semantic retrieval is based on embedding similarity:

$$\mathrm{sim}(q, d) = \frac{q \cdot d}{\|q\|\|d\|} \tag{3.2}$$

where q is the query embedding and d is a recipe embedding. Top-k nearest matches are returned from Milvus and enriched with recipe details through backend API calls.

3.6.1 Response Generation and Audio Streaming

The response layer separates text planning and audio playback:

- LLM/path handler produces response text (or structured recipe blocks),
- TTS module splits long responses into sentence chunks,
- chunked audio player streams chunks and supports pause/resume.

This design reduces perceived latency because playback can begin after the first generated chunk rather than waiting for full-response synthesis.

3.7 Session and State Management

Session data tracks recipe ID, current step, spoken sections, and conversation turns. Persistence uses Upstash Redis when configured; otherwise, the system falls back to in-memory state. This preserves functional continuity in both connected and degraded modes.

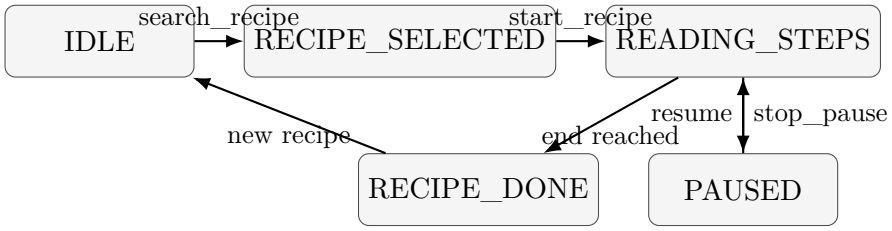

Figure 3.4: Simplified runtime conversation/state progression.

3.8 Error Handling and Fallbacks

The design implements cascading fallbacks:

- ASR fallback: Deepgram → local Whisper.
- Intent fallback: rule-based → LLM classification.
- Storage fallback: Redis → in-memory dictionary.
- Response fallback: structured JSON parsing → safe plain-text conversion.

3.9 Implementation Notes

The project implementation emphasizes readability and modularity:

- each subsystem is encapsulated in `modules/`,
- runtime orchestration is centralized in `src/main.py`,
- data contracts are documented via schema and API documentation files,
- algorithm explanations and design rationale are maintained in companion docs.

This structure supports faster debugging and easier incremental enhancement by future contributors.

Chapter 4

Results and Discussion

4.1 Implementation Status

The project is implemented end-to-end and organized into reusable modules. The runtime loop integrates wake-word detection, ASR, intent classification, retrieval/navigation, LLM response generation, and TTS playback with interruption handling.

Table 4.1: Pilot dataset and artifact summary from current repository state

Artifact	**Count**	**Observation**
`data/recipes.csv`	100 rows	Recipe metadata entries (100 unique recipe IDs)
`data/chunks.csv`	100 rows	Recipe navigation chunks used at runtime
`data/searchable_text_for_embeddings.csv`	100 rows	Embedding-ready searchable text
`data/food_dictionary.csv`	100 rows	Ingredient dictionary per recipe
`ASR_text/`	233 files	Stored transcriptions from voice interactions
`voice_recordings/`	165 files	Captured WAV command recordings
`tts_generated_speech/`	656 files	Generated speech chunks and responses

4.2 Module Completion Snapshot

Table 4.2: Module-level completion summary

Module	Status	Outcome
`wakeword.py`	Complete	Continuous wake-word detection with event signaling
`vad.py`	Complete	Silence-based endpoint recording and WAV output
`deepgram_asr.py` + `asr.py`	Complete	Cloud ASR with local fallback path
`intent_classifier.py`	Complete	Rule+LLM hybrid classifier with entity extraction
`retriever.py`	Complete	Semantic recipe search and detail fetch integration
`navigator.py`	Complete	Deterministic step/ingredient navigation logic
`session_manager.py`	Complete	Redis-backed session with in-memory fallback
`tts.py` + `audio_player.py`	Complete	Chunked TTS generation and interruptible playback

4.3 Observed Pipeline Performance

Based on project documentation and runtime instrumentation in `src/main.py`, the interaction loop demonstrates practical responsiveness for conversational cooking.

Table 4.3: Typical operation-wise latency ranges

Operation	Typical time	Notes
Wake-word frame processing	< 50 ms/frame	Continuous background thread
VAD recording window	3–5 s (user-dependent)	Ends after sustained silence
ASR (Deepgram path)	1–3 s	Local Whisper fallback available
Rule-based intent classification	10–50 ms	LLM fallback used on low confidence
LLM fallback / generation	0.5–3 s	Used for ambiguous or conversational cases
Semantic retrieval (Milvus)	100–500 ms	Top-k nearest recipes
TTS first audible chunk	$\sim$1 API RTT to first chunk	Streaming playback starts early

4.4 Interaction Behavior Analysis

The observed behavior across runtime traces indicates:

- deterministic handling for navigation commands ("next", "repeat", "go to step"), primarily due to rule-based high-confidence intent matches,
- smoother perceived response because playback starts with first TTS chunk instead of full-response buffering,
- practical interruption tolerance: wake events can pause active narration and trigger a fresh command cycle.

4.5 Example End-to-End Runtime Trace

Table 4.4: Representative interaction trace

Step	**User/Runtime action**	**System response**
1	User says "Hey Cook"	Wake-word event set; assistant pauses any active playback
2	User asks "How do I make pasta?"	ASR transcript generated and intent classified as `search_recipe`
3	Retriever executes semantic search	Top recipe loaded; session initialized with recipe metadata
4	User says "start cooking"	Ingredients section and first step narrated
5	User says "next step" while audio playing	Playback interrupted; navigator advances step index and speaks next step
6	User asks substitution question	LLM QA path uses recipe context and returns concise answer

4.6 Functional Outcome

Table 4.5: Functional coverage achieved in the prototype

Capability	Status	Evidence path
Wake-word interrupt loop	Implemented	`modules/wakeword.py`, `src/main.py`
Recipe search and start flow	Implemented	`modules/retriever.py`, `modules/navigator.py`
Step-by-step navigation	Implemented	`get_next_step`, `jump_to_step`, repeat handlers
Contextual question answering	Implemented	`modules/llm.py`, intent `question` path
Session persistence with fallback	Implemented	`modules/session_manager.py`
Chunked speech playback with pause/resume	Implemented	`modules/tts.py`, `modules/audio_player.py`

4.7 Failure-Mode and Recovery Summary

Table 4.6: Observed failure classes and fallback behavior

Failure condition	Primary impact	Recovery behavior
Deepgram ASR unavailable	Transcript unavailable from cloud path	Local Whisper ASR fallback maintains command handling
Low-confidence intent parse	Potentially wrong dispatch route	LLM fallback classifier resolves ambiguous utterance
Redis unavailability	Session persistence loss risk	In-memory session fallback preserves current runtime continuity
Invalid structured LLM output	Response structure parsing failure	Safe fallback to plain-text response generation
Audio interruption during long response	User waits or loses context	Chunked player pause/resume with session-based continuation

4.8 Scalability Considerations

Although the current pilot uses 100 recipes, the chosen architecture naturally supports growth:

- embedding retrieval can scale with vector index size,

- stateless module interfaces support parallelization and service decomposition,
- externalized session storage enables multi-instance runtime scaling.

Table 4.7: Projected scaling considerations

Dimension	**Current state**	**Expected engineering action for larger deployment**
Recipe corpus size	100 recipes	Increase embedding/index build cadence; optimize vector search parameters
Concurrent users	Prototype single-session style	Introduce session sharding and queue-based TTS workers
ASR/TTS API throughput	Single interaction loop	Add request throttling, retries, and provider-level failover policy
Observability	Console-level runtime logs	Add structured logging, metrics dashboards, and trace IDs

4.9 Discussion

The system demonstrates a strong integration baseline for smart kitchen use-cases. The most significant strength is architectural modularity: each subsystem can be replaced independently (e.g., ASR provider, LLM model, or storage backend) without redesigning the full pipeline.

The implementation demonstrates stable end-to-end behavior on the pilot dataset and highlights a practical blueprint for domain-specific conversational assistants that need real-time command handling and context-aware narration.

4.10 Validity Considerations

The reported observations should be interpreted with the following scope boundaries:

- latency ranges depend on network and API conditions,
- retrieval relevance depends on embedding model/domain fit,
- conversational quality depends on LLM service behavior and prompt constraints.

These factors are common in voice-assistant research prototypes and motivate future controlled benchmarking.

4.11 Conclusion

The **Smart Kitchen Assistant Model Development** project successfully delivers an operational voice-first cooking assistant with retrieval, navigation, conversational support, and robust fallback behavior. The developed architecture is suitable as a research and engineering baseline for next-generation smart kitchen interaction systems.

Bibliography

[1] Picovoice. Picovoice porcupine wake word engine documentation, 2026. Accessed 2026-04-16.

[2] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Robust speech recognition via large-scale weak supervision. *arXiv preprint arXiv:2212.04356*, 2023.

[3] Deepgram. Deepgram api documentation: Speech-to-text and text-to-speech, 2026. Accessed 2026-04-16.

[4] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of EMNLP-IJCNLP*, pages 3982–3992, 2019.

[5] Feifei Han et al. Milvus: A purpose-built vector data management system. *Zilliz Technical Documentation*, 2021. Accessed 2026.

[6] Google AI for Developers. Google gemini api documentation, 2026. Accessed 2026-04-16.

[7] Upstash. Upstash redis documentation, 2026. Accessed 2026-04-16.